An Investigation and Technical Demonstration of a Course Management System for Golfers

Ryan Shaw
19692918

19692918@students.lincoln.ac.uk

School of Engineering and Physical Sciences

College of Health and Science

University of Lincoln

Submitted in partial fulfilment of the requirements for the
Degree of BSc(Hons) Computer Science

Supervisor: Dr. Mark Doughty

April 2025

Acknowledgements

Firstly, I want to thank everybody who has supported me through my journey through University particularly my parents as well as my friend group who have helped me through this time.

I would also like to extend my thanks to Market Rasen Golf Club where the majority of testing took place. Themselves and their members have been really accommodating throughout this phase of the study to allow sufficient data to be collected as well as the University of Lincoln Golf Society who also aided in data collection.

Abstract

Golf has long been a game of improving scores with improving technology aiding the player to achieve their goal. With the rise of internet connected mobile devices and the Global Positioning System there has been the idea to track your shots in more detail than was possible beforehand. However, it is not provably known if utilising GPS improves your scoring itself due to the variability of golf itself. The aim of this study is to investigate if a GPS scoring system improves a players score with a high confidence margin. This has been done through the development of a test GPS distance measuring application within a React web application before testing the scores of players using and not using the application. The results of this study suggest that utilising GPS distance measuring improves scores on Par 3's with a 99.53% confidence margin, Par 4's with a 99.99% confidence margin and Par 5's with a 99.99% confidence margin. As a result of this study it implicates that players should always be utilising GPS distance measuring where allowed and that claims made by commercial companies around the advantage of using their GPS applications are measurably true.

Table of Contents

Introduction 1

1.1 Background 1

1.2 Motivations 2

1.3 Aims and Objectives 2

Literature Review 4

2.1 Theoretical Approaches 4

2.2 Analysis of current factors 4

2.3 Informed Decision Making 5

Requirements Analysis 7

3.1 Functional Requirements 7

3.2 Non-Functional Requirements 8

Design & Methodology 9

4.1 Project Management 9

4.1.1 Software Development Methodology 9

4.1.2 Toolsets and Environments 13

4.1.3 System Design 13

4.1.4 Functionality Testing 15

4.2 Risk Analysis 16

4.3 Participant Recruitment 17

4.4 Ethical Procedures 17

4.5 Study Design 17

4.6 Statistical Analysis 18

Implementation 19

5.1 React App 19

5.2 Components 25

5.3 Utils 32
5.4 Hosted 33
Results & Discussion 36
Conclusion 42
References 43
Appendix A 46

List of Figures

1. Agile Methodology 5
2. Gantt Chart w/c 23/09/24 – 18/11/24 11
3. Gantt Chart w/c 27/01/25 – 05/05/25 12
4. Example Course Management screen 1 (Shaw, 2024) 14
5. Example Course Management screen 2 (Shaw, 2024) 14
6. index.html 19
7. main.tsx 20
8. Constants definitions 20
9. Constant User Positions 21
10. Location Handing 22
11. Initial Screen code 22
12. Selection Screen code 23
13. Map Screen code 24
14. Manual Data Types 24
15. data.ts 25
16. SelectionButton.tsx 26
17. LocationList.tsx 27
18. Pin and Map Props 27
19. MapEvents 28
20. Map Render 29
21. Map HTML 30
22. Weather Request 31
23. Parsing and Return 31
24. Haversine Equation code 32
25. Initial Screen 33
26. Course Selection Screen 34
27. Map Screen 35
28. Linear Regression of Scores to Par 37
29. Par 3 Without GPS Histogram 38
30. Par 3 With GPS Histogram 38
31. Par 4 Without GPS Histogram 39
32. Par 4 With GPS Histogram 39
33. Par 5 Without GPS Histogram 40
34. Par 5 With GPS Histogram 40
35. Consent Form 46
36. Participant Information Sheet 47
37. Participant Information Sheet 48

List of Tables

1. Risk Analysis 5
2. Sample Data 36
3. Results from Mann-Whitney U Test 41
4. Results from Welch's T Test 41

Chapter 1

Introduction

1.1 Background

Golfers have a primary aim of taking the least amount of shots to get a ball of diameter no smaller than 1.680 inches and weighs no more than 1.620 ounces avoirdupois (R&A, 2022) into a hole that is at least 4 inches in diameter and at least 4 inches deep (R&A, 2022). This goal creates strategy about how best to achieve this as typically there is a great difficulty in succeeding in one shot from the teeing area.

Golfers have many challenges in shooting a low score due to it being the only sport with no specific dimensions (Golf's Grand Design, 2012). This forces strategic decisions which are individual, date and time specific as ability, weather and temperature are all factors.

This study aims to demonstrate that a golf course management system for players measurably decreases scores throughout a round of golf. The nature of this study requires the creation of an artefact which will allow users to measure the distance from them to a specific point of their choosing. An artefact of this nature requires the use of GPS as it can get the approximate location of the user to a great precision (GPS.gov, 2019). A top-down view of the course will also be required which will be achieved via satellite images. There also needs to be a system with courses on which will either be present locally in the artefact or it can be accessed through a connection to a central server (Sachan, 2024). To create an integrated system across different devices the project will require a tech stack (Heap, n.d.), which can either be a pre-created tech stack or a custom-made tech stack for the bespoke requirements of this project (Raroque, n.d.).

It is also known that the way in which you feed golfers information can change how someone plays the sport as well as who feeds them the information (Jowett and Zhong, 2016). This has already been studied extensively so this study can utilize that and figure out how to electronically provide this information in the best possible way as it has traditionally been done through caddies who have training on how to manage a round of golf for someone else quickly figuring out

how to convey information to them for the maximum benefit (Carey, Stone and Lavallee, 2021). (Shaw, 2024a)

1.2 Motivations

The rationale for this study is that generally the more information you give golfers the better scores they shoot as they can make more informed decisions especially with distances (Dithurbide and Neyedli, 2019). This can also be an evolution of the continuing rise of science and technology in golf which has been moving forwards at all levels of the sport in recent years (Haskett, 2022). The project also allows for an insight into Gamification of sports, due to the line between a sport and a game is already incredibly fuzzy this is more combining video games with real life sports to improve performance (Tóth and Lógó, 2018).

The idea for this study has come from my extensive time playing golf on a course as well as on a simulator so there is inspiration taken from systems I have used like ShotScope (Shotscope.com, 2023) and GSPro/Foresight Sports (ForesightEurope, 2024), taking features from both as well as taking a hardware idea I have seen from MotoCaddy (Motocaddy, 2024) and combining all three ideas together to create something new as an evolution. (Shaw, 2024a)

1.3 Aims and Objectives

After some careful consideration around the wording of my Aim for the study it has been concluded that the aim of this study is “To develop a course management application for golfers which can improve their score on a hole”. This overall aim lays out the basis of what will be created throughout this project, but it allows for different ways to go about achieving the aim which is laid out in my objectives.

The objectives have been distilled into 4 key features to give a level of intuitive ease of use for the participant as well as providing the key features to be able to aid the golfer in the most basic sense while following all the rules of golf. All information presented must be publically available and weather conditions must not be measured by any device live.

To create an artefact which would be able to provide test data to analyse to determine if any changes in scores are statistically significant it is required to:

Calculate the distance from current GPS location to a user defined point (yards or meters).

Have multiple courses data stored centrally which can be downloaded on request if needed.

Show a satellite image of a selected golf hole.

Display weather forecast information including the Wind Direction, Strength and Temperature. (Shaw, 2024a)

Chapter 2

Literature Review

2.1 Theoretical Approaches

There have been a few papers researching the absolute statistical best way to play a hole focused on score minimization as the foundational standard of rational behaviour (Mack, 2015). There are also purely statistical research which has been per-formed and directly lead to the Strokes Gained metric which is applied to the highest levels of golf (Broadie, 2011).

Broadie lays out a plan on how to apply the insights from Strokes Gained at the top level into the amateur level in his book Every Shot Counts: Using the Revolutionary Strokes Gained Approach to Improve Your Golf Performance and Strategy (Broadie, 2014). However, this has been critiqued for the simplicity of the suggested by Mack (2015) stating that suggestions made by Broadie in Chapter 8 “Tee-to-Green Strategy: How Data and Optimization Can Lower Your Score” where he attempts to coach the reader through “several strategic situations so you can learn how to develop a game plan to shoot lower scores”.

Mack’s criticism stems from the data being used to gain this insight coming purely from top-level professional golf “These suggestions are well supported and I do not necessarily disagree with his suggestions but I find this approach problematic because all of his suggestions rely on insights from data on the actions and outcomes of golf shots hit by professionals who play at the highest level of the game” (Mack, 2015). (Shaw, 2025)

2.2 Analysis of current factors

While the discussion regarding lowering scores talks about the theoretical best decision to make in a specific situation there is also research into how golfers are currently coming to their decisions. The most comprehensive study coming from Shapcott (2021) and Published by the United States Golf Association. This qualitative analysis of the factors of decision making finds that from the tee on Par 4’s that 84.12 per-cent of golfers surveyed hit the Driver off the Tee all the time with 90 percent indicating their shot was of Good Quality. Shapcott addresses an issue with this data that it was collected from individuals recounting

their round so it is suggested that they are not accurately recalling the quality of their tee shots. Shapcott identifies that this is consistent with Barzeki & Wedzik (2014), who argue that driving distance is more important than short game proficiency, and Broadie (2008) who argues that playing approach shots from the rough has less of a detrimental effect for amateur players than professional players.

Another factor is revealed when looking at Shapcott's (2021) data for Club Selection from the Tee on Par 5's, this is due to the percentage hitting Driver increases to 91.36 percent. This is hypothesized to be due to the longer distance of the whole requiring more distance from the tee or a belief that on a Par 5 a shot can be recovered. These findings are consistent with Elmore and Urbaczewski (2020) who found that players scored lower when a hole was changed from a Par 5 to a Par 4 without changing the distance of the hole, Elmore and Urbaczewski concluded that this was due to players being more Risk Adverse on Par 4's. Shapcott (2021) suggests that that rationale could explain her findings as to why players are more likely to hit a Driver on a Par 5.

These conclusions that have been drawn suggest there is a perfect spot between Distance and Risk Aversion which will depend on each player's specific game and the condition of every hole played. (Shaw, 2025)

2.3 Informed Decision Making

Unfortunately studies into the decision making process in golf have usually had some flaws present which are noted by Gnagy et al. (2015). Who note that studying human decision making is notoriously complex endeavour with golf focused research generally investigating the presence of framing effects or testing the correctness of strategic decisions by Sachau, Simmering and Adler (2012) and Kirschenbaum, O'Connor and Owens (1999) respectively. Gnagy et al. (2015) note that despite the importance of this area that research is scant but from their study they note that how individuals play is directly influenced by assessment of the situation an individual finds them-selves in and is not pre-meditated. But that study data so far has the flaw of usually being qualitative data and having been collected after a round has been completed leading to potential recency and confirmation bias.

As research in other fields suggests that a more informed decision is more likely to be closer to the optimal decision such as a study by Clark et al. (2024) which looked at evidence-based decision making in a healthcare setting and found that

when a decision was taken where more information was available outcomes improved. (Shaw, 2025)

Chapter 3

Requirements Analysis

From the Basic Aims and Objectives of the study there are several functional and non-functional requirements that the artefact must satisfy to be able to perform its role as a part of data collection with user testing.

3.1 Functional Requirements

Display of current gps location. The artefact should display the users current location in real-time on an interactive map so the user can see where they are in relation to the continuing golf hole and so they can orientate themselves with where they should go with the next shot.

Display distance to a selected point. The artefact should allow the user to select a point on an interactive map and then relay the distance to that specific point back to the user. This distance should be measured from their current location onwards to allow for a decision to me made by the user. The point should update every time the user selects a new point on the interactive map.

Multiple course selection. Multiple courses should be selectable by the user to reflect the different locations where testing will be taking place. This selection of courses should be ordered by the closest course so the most logical option for when testing is present at the top as that will be the users location. This feature is also present in many current distance measuring tools for ease of use.

Display of current weather forecast. The current weather should be displayed. There can be no measurement of the weather live as this is classified as removing judgement and thus incurs a penalty according to the rules of golf. However, a weather forecast for the location can be provided. Temperature, Humidity and Wind Direction/Strength can all effect ball flight and distances. A higher temperature, lower humidity and a strong tailwind will all cause a golf ball to fly further even though heat and humidity are directly linked. Providing a Forecast for the location of the course will allow a player to determine their potential distance more accurately potentially aiding their scoring.

Ethical considerations. When performing research with human participation ethical considerations should be taken into account such as participation and researcher safety, equipment condition and privacy. All personal data that is collected also needs to be stored confidentially and securely.

3.2 Non-Functional Requirements

Intuitive UI/UX. Throughout the artefact the user should be able to navigate to the desired section of the program, primarily the interactive map that is present for the user to be able to perform any testing. How they choose a point should also be clear with the distances also very clearly visible and use should require no instruction from the testing co-ordinator.

Battery Efficiency. As the artefact requires being on a mobile device for testing to be performed and the course of the testing encompassing playing 16 holes of golf there needs to be a level of battery efficiency so it can be used for a total of up to 6 hours without break. This can either be considered natively if the artefact is a standalone application or it can be abstracted away if another application is used to host the artefact e.g. web browser.

Accuracy. Due to the use of gps and the nature of its use in this artefact there is a required level of accuracy of 2 to 3 metres. This is needed as for a distance measurement to be accurate enough for a club and shot type to be chosen this gives a range of 6 yards back to front which should be the decision area for each golfer to play a shot.

Accessibility. Golfers should be able to read the information present even if they have some difficulties receiving the information. This should include a contrast of 7:1 for all normal text present. If the artefact is placed inside a web browser there are tools present within browsers to act as a screen reader for those who struggle to see but the nature of manually selecting a point could present challenges to achieving this goal for everyone. This study will specifically follow the Web Content Accessibility Guidelines (WCAG) 2.1 which were published on 12th December 2024 (W3.org, 2024).

Chapter 4

Design & Methodology

4.1 Project Management

4.1.1 Software Development Methodology

For the development of the artefact an agile methodology has been chosen due to its cycle of continuous testing. Its stepped process of Designing, Planning, Developing and Testing is perfect for creating each feature independently allowing for functionality to be layered on top of the application repeatedly. This is due to agile being an iterative development process focussed on small deliverables to create a greater product (NVISIA, 2020).

Figure 1. Agile Methodology (NVISIA, 2020)

As this is a development team of one person meetings are not needed but deadlines for each feature can be imposed still to help speed up and keep track of development so the artefact can be delivered on schedule so it can facilitate the testing to help answer the hypothesis.

Throughout the Agile process a time management process is adhered to as laid out before development begins. This gives firm deadlines to work towards for

different sections of the study as a whole. This is laid out in a Gantt Chart on the following page.

GANTT CHART TEMPLATE by Vertex42.com
https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html

TASK	ASSIGNED TO	PROGRESS	START	END
Project Proposal				
Write Project Overview	Ryan Shaw	100%	23/09/24	31/10/24
Collect References	Ryan Shaw	100%	23/09/24	31/10/24
Establish Aims and Objectives	Ryan Shaw	100%	23/09/24	31/10/24
Create Gantt Chart	Ryan Shaw	100%	23/09/24	31/10/24
Analyse Potential Risks	Ryan Shaw	100%	23/09/24	31/10/24
Literature Review	Ryan Shaw	100%	23/09/24	30/01/25
Technology Investigation				
Investigate Primary Development Language	Ryan Shaw	100%	01/11/24	30/01/25
Investigate UI/UX	Ryan Shaw	100%	01/11/24	30/01/25
Design Database Structure	Ryan Shaw	100%	01/11/24	30/01/25
Data Protection Investigation for User Data	Ryan Shaw	100%	01/11/24	30/01/25
Investigate Satellite/Aerial Photography Availability	Ryan Shaw	100%	01/11/24	30/01/25
Development				
Build Database Structure	Ryan Shaw	100%	31/01/25	28/02/25
Add Mutiple Courses with Hole Bounding to Database	Ryan Shaw	100%	01/03/25	01/05/25
Get GPS to Selected Point Distance	Ryan Shaw	100%	31/01/25	01/05/25
Iterate/Test UI/UX	Ryan Shaw	100%	15/11/24	01/05/25
Connect the Different Structures Together	Ryan Shaw	100%	31/01/25	01/05/25
Report Writing				
Enter Investigation Information	Ryan Shaw	100%	01/11/24	01/05/25
Detail Design Information	Ryan Shaw	100%	01/11/24	01/05/25
Analyse Challenges and Development Resources	Ryan Shaw	100%	01/11/24	01/05/25
Add Completed Code Documentation	Ryan Shaw	50%	01/11/24	01/05/25
Evaluate Project Success	Ryan Shaw	25%	02/05/25	08/05/25

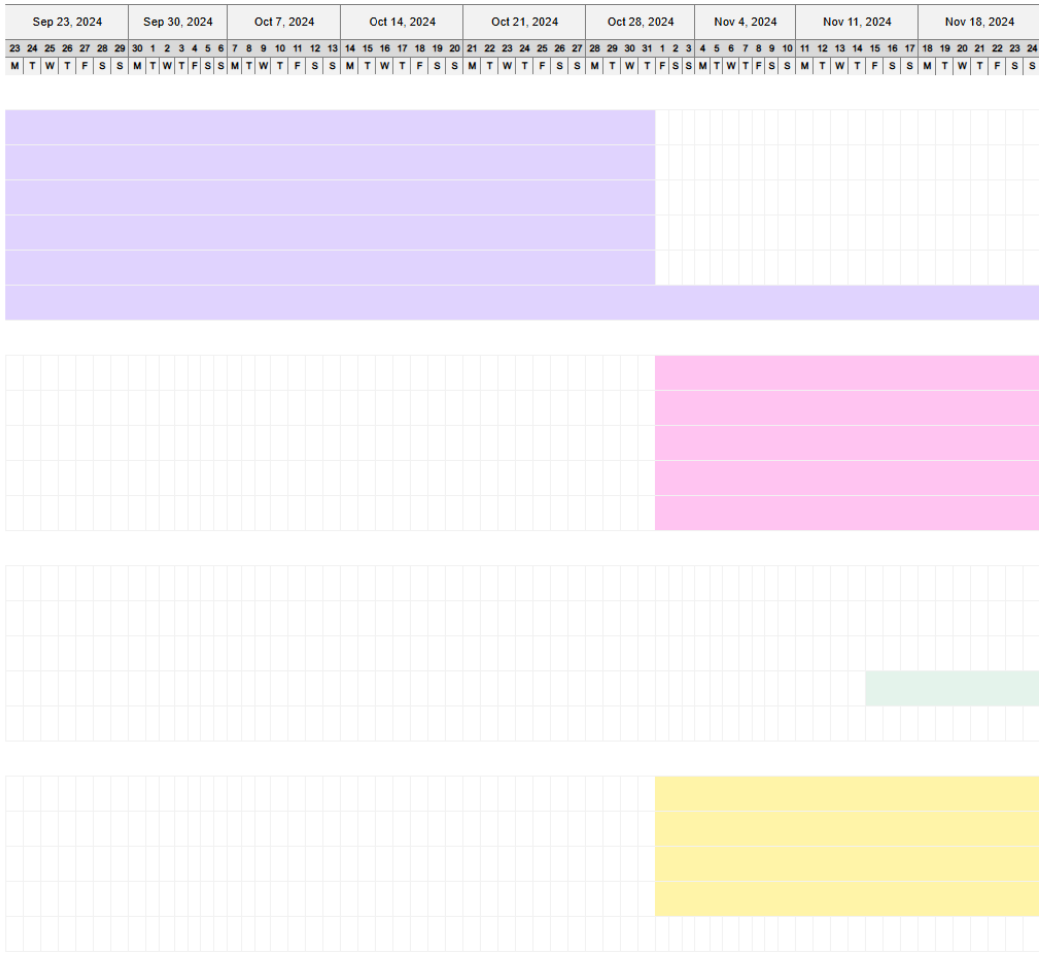

Figure 2. Gantt Chart w/c 23/09/24 – 18/11/24 (Shaw, 2024a)

GANTT CHART TEMPLATE by Vertex42.com
https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html

TASK	ASSIGNED TO	PROGRESS	START	END
Project Proposal				
Write Project Overview	Ryan Shaw	100%	23/09/24	31/10/24
Collect References	Ryan Shaw	100%	23/09/24	31/10/24
Establish Aims and Objectives	Ryan Shaw	100%	23/09/24	31/10/24
Create Gantt Chart	Ryan Shaw	100%	23/09/24	31/10/24
Analyse Potential Risks	Ryan Shaw	100%	23/09/24	31/10/24
Literature Review	Ryan Shaw	100%	23/09/24	30/01/25
Technology Investigation				
Investigate Primary Development Language	Ryan Shaw	100%	01/11/24	30/01/25
Investigate UI/UX	Ryan Shaw	100%	01/11/24	30/01/25
Design Database Structure	Ryan Shaw	100%	01/11/24	30/01/25
Data Protection Investigation for User Data	Ryan Shaw	100%	01/11/24	30/01/25
Investigate Satellite/Aerial Photography Availability	Ryan Shaw	100%	01/11/24	30/01/25
Development				
Build Database Structure	Ryan Shaw	100%	31/01/25	28/02/25
Add Multiple Courses with Hole Bounding to Database	Ryan Shaw	100%	01/03/25	01/05/25
Get GPS to Selected Point Distance	Ryan Shaw	100%	31/01/25	01/05/25
Iterate/Test UI/UX	Ryan Shaw	100%	15/11/24	01/05/25
Connect the Different Structures Together	Ryan Shaw	100%	31/01/25	01/05/25
Report Writing				
Enter Investigation Information	Ryan Shaw	100%	01/11/24	01/05/25
Detail Design Information	Ryan Shaw	100%	01/11/24	01/05/25
Analyse Challenges and Development Resources	Ryan Shaw	100%	01/11/24	01/05/25
Add Completed Code Documentation	Ryan Shaw	50%	01/11/24	01/05/25
Evaluate Project Success	Ryan Shaw	25%	02/05/25	08/05/25

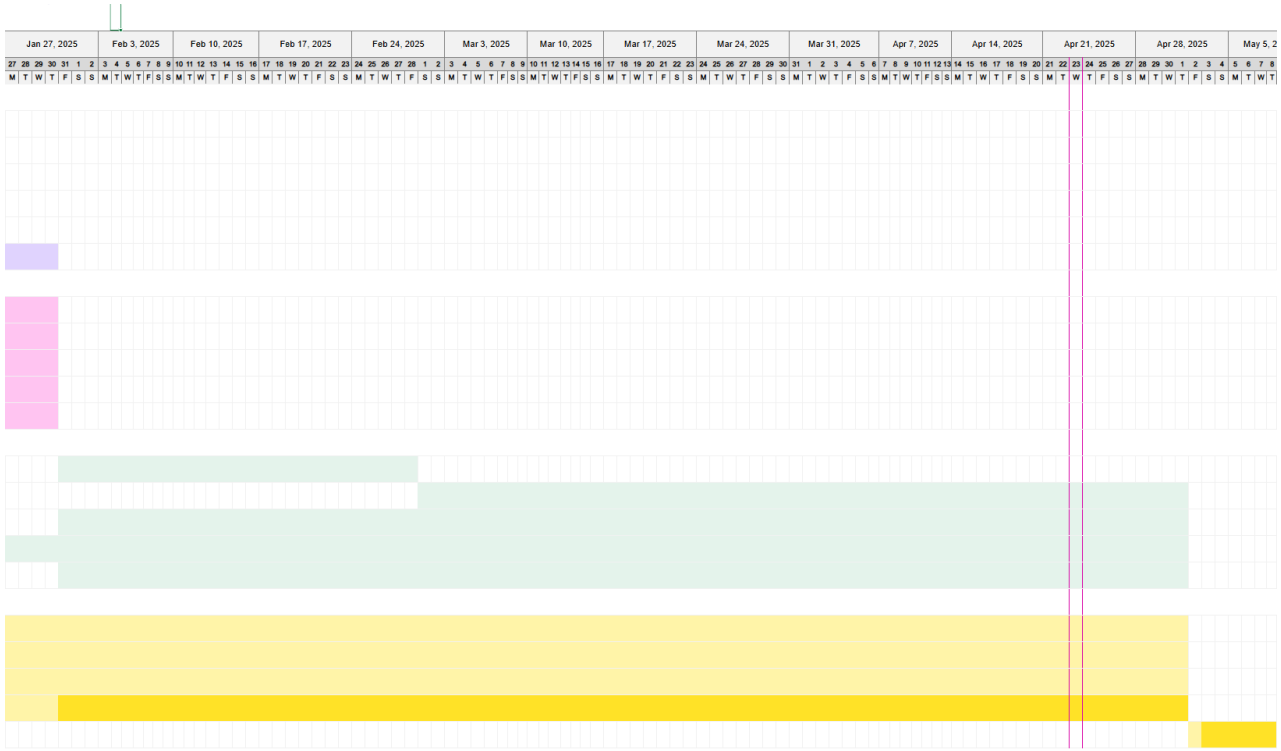

Figure 3. Gantt Chart w/c 27/01/25 – 05/05/25 (Shaw, 2024a)

4.1.2 Toolsets and Environments

To aid the creation of the artefact to ensure it can fulfil the non-functional requirements of Battery Usage and Accessibility the environments used is a Web Application as responsive design can be used to run on a mobile device, Web Browsers also have built-in accessibility tools such as screen readers as well as reader modes, while the development itself can take place utilizing React, Vite and Typescript to integrate and optimize performance. As such React is declarative so the state of a component can be changed with React updating the DOM meaning the design and debug of the User Interface is easier (GeeksforGeeks, 2022).

Vite was also chosen to be used because it allows visualization of changes in real time reducing development time as the process will be iterative in its nature. Vite optimizes code when it builds allowing for a better experience for users due to the lower execution time and resource usage (Caio, 2024). Typescript is supported which allows for static typing and annotations on top of traditional JavaScript. It is a high level language which was developed by Microsoft for large web applications due to a belief that the lack of in-built module system which meant there were a lack of elegant ways to "produce self-contained blocks of code that present well-defined interfaces to developers" (Staff, 2012).

A key consideration when choosing any web based tech stack is the hosting available. It was decided GitHub Pages would be used as this allowed for native version control with no fee for hosting the website itself (GitHub, n.d.). This choice meant that the web app had to be a static site with no server-side processing limiting some features possibility and potential implementations of others. (Shaw, 2025)

4.1.3 System Design

The UI/UX design of the artefact must be in keeping with the functional and non-functional requirements of the overall system. As such the basic objectives of this project are: Calculate the distance from current GPS location to a user defined point (yards or meters), Have multiple courses data stored centrally which can be downloaded on request if needed, Show a satellite image of a selected golf hole.

As such these requirements have been considered and designs have been made with inspiration being taken from Shaw (2024b) who had undertaken a user experience design process to develop ideas and program flows which were considered intuitive to the user.

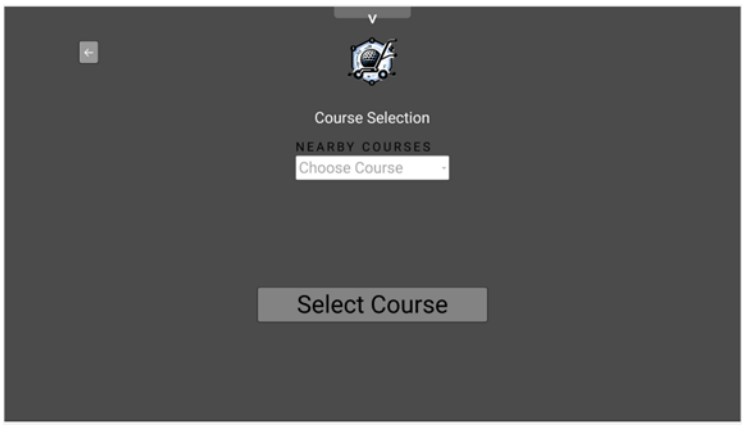

Figure 4. Example Course Management screen 1 (Shaw, 2024b)

Figure. 5. Example Course Management screen 2 (Shaw, 2024b)

As the main focus of this study is in the technical GPS aspects and data analysis of the results to prove/disprove a hypothesis after testing the decision was taken to utilize AI tools to develop the interface of the application so more time could be placed into the functionality and testing. For this use of AI it was decided to use bolt.new as it can be connected directly to GitHub and the technologies used can be defined in the prompt with them being understood (bolt.new, 2024).

The prompts utilized were: “I want to use React and Vite with Typescript. I would like a web app which has the background colour #4B4B4B and a foreground colour of #D9D9D9. It should have a selection button which then takes you to a place to select an option from a list. And when an option from the list is selected there should be an interactive map of a small area. Can the selection be for Carholme Golf Course and Lincoln Golf Club. With images being used in the buttons as well. Can it be changed to a satellite map in which the user can select a point on the map themselves returning the GPS co-ordinates and return the GPS co-ordinates of the user as well. Can the distance from the user to the selected point be displayed with a straight white line and in yards having used the Haversine Equation. Can the distance be displayed next to the

white line and not under the point itself. Can the selection button show courses in order of closest to current location to furthest."

These prompts gave a basic web application with very limited and bug filled functionality to be iterated upon and improved. The basic colour scheme was given but beyond that no other design elements were communicated. A basic selection of two golf courses were given to allow for a demonstration of a method of how to achieve the course list as well as taking the co-ordinates of them to demonstrate how to order the list from closest to furthest. This functionality required the Haversine Formula which calculates the distance around a sphere between 2 points on the surface of the sphere (Robusto, 1957).

An initial problem with the provided generated application was the use of getCurrentPosition which uses GPS to return the co-ordinates of the users' location. However, this is a single retrieval of the co-ordinates which work when the map is first viewed but it doesn't update unless totally reloaded which is not optimal for this type of application where it will be open as the user moves down a golf hole (MDN Web Docs, n.d.a). This lead to a change where watchLocation is used instead as it returns co-ordinates depending on the developer defined time between retrieval. This is set at every 0 milliseconds currently as the memory is also cleared immediately after it is plotted on the map to prevent a memory leak (MDN Web Docs, n.d.b) which could also potentially drain the battery of the device it is being utilised on.

Sharpley Golf as well as Market Rasen and District Golf Club were also been added to the course selection screen to aid with user testing as well as revealing a further development of only showing the closest golf courses as well as demonstrating the technical way to add extra course to the system itself. (Shaw, 2025)

4.1.4 Functionality Testing

Functionality Testing was performed throughout the development process following the principles of Agile development. Each feature to be added was first designed in how it would function and look on the artefact. Due to updating the live version of the artefact on GitHub Pages the Vite live version was used to ensure the design was responsive to multiple screen dimensions before the features functionality was tested.

To undertake this functionality testing myself and only myself took the live version of the artefact and moved around to test the live gps position on my mobile device through a web browser as well as testing the distance measuring capabilities at the same time. These distances were then compared to those that Google Earth gave in their distance measuring (Google, 2019). When the

implementation of these features were satisfactory the process of agile development continued moving onto the next feature until the artefact could help provide the data required from the user testing.

4.2 Risk Analysis

Hazard	Risk	Likelihood	Mitigation
A suitable Tech Stack can't be found for the project	Prevention of Development or a Disjointed collection of technologies which struggle to communicate to each other	Moderate	A custom Tech Stack can be created attaching languages and technologies together manually
Licensing of Satellite Data	Inability to license satellite data could lead to no maps being available for the user to view the hole	Low	Find any publicly available land data or use Open-Source information to create the data or try and find Aerial Imagery instead
Insufficient User Functional Testing	Lack of feedback and iterations of the project due to the lack of user testing taking place	Moderate-High	Constant self-testing to ensure functionality and separate concurrent UI/UX testing
Insufficient Study Participants	Lack of data collected preventing meaningful conclusions being drawn from the data	High	Use of self as test subject. Utilisation of skewed distribution metrics to provide meaningful analysis

Table 1. Risk Analysis (Shaw, 2024a)

There were several possible risks throughout the development of the artefact as well as throughout the testing for the study. They have been documented in the figure above listing the Hazard, Risk, the Likelihood of it occurring and the mitigation to allow the study to continue.

4.3 Participant Recruitment

Research of this nature requires participants to complete testing where the data they generated shall be analysed to answer the research question. This data will be analysed in Chapter 6 of this report into the study. Participants were recruited from the University of Lincoln Golf Society as prior experience playing golf was required and an official handicap was used to verify this. All participants were above the age of 18 and signed consent forms verifying this. All testing took place at golf courses local to Lincoln and their use was paid for through green fees.

4.4 Ethical Procedures

During testing upholding ethical procedures was of upmost importance. The only personal information stored was an individual's email address with no other identifying features only for the purpose of informing them about the results of the study after it has been completed. Providing this was of the participants own will and they were under no obligation to provide this if they did not wish to hear about the results of the study. All Participants were provided with a Participant Information Sheet and a Consent Form (both viewable in Appendix 1). They also showed their official handicap from an official organisation (e.g. England Golf) and this was recorded for the study to allow for proper analysis. A players handicap is not considered personal data as it cannot be linked to them while if they has provided a membership number this would be considered personal information. All Personal Information was stored securely within the University's OneDrive which requires password access.

4.5 Study Design

When the participant had agreed to take part a time and a date was organised for it to take place due to the longer time frame and usual location of golf courses requiring moderate logistical planning to take place. Upon deciding the location a selection of holes were chosen semi-randomly by the researcher. All players would play 8 holes twice for a total of 16 holes played. Each participant played at least 2 par 3's, 2 par 4,s and 2 par 5's with the last 2 holes they were tested on were completely randomly chosen from the remaining holes. If a participant started feeling any physical, mental or emotional discomfort they were asked if they would like to continue participation and if they did not testing ended immediately with all data they were producing being nullified

and not recorded. An individual could partake in the study multiple times as the focus on the research is the relationship between Scoring, Par and Use of a GPS device and not related to Human factors. Each time they participated they signed a new Consent Form and re-read the Participant Information Sheet.

Once the testing was complete they were free to complete their paid round, as is their right from paying a green fee, if they would like or they could end there.

4.6 Statistical Analysis

All collected data from this study was Quantitative with the aim of analysing it statistically. The ways of statistically analysing the data was chosen before any was collected with Linear Regression of the scores, a Mann-Whitney U Test and a Welch's T Test to provide information on the change of score compared to Par and use of the GPS, the probability that the data was statistically similar with an assumed common variance and the probability that the data was statistically similar without an assumed common variance.

All data will be analysed in this manner with nothing removed even if it seems anomalous at first glance as its occurrence has been verified by the researcher. The results of the data shall be visualised into graphs and tables to better help with their visualisation.

Chapter 5

Implementation

5.1 React App

The main body of the GPS distance measuring application is a React application. As such the application the index.html file contains very little but points towards the typescript file which contains the initial imports.

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Course Management</title>
    <link rel="icon" type="image/x-icon" href="./favicon.ico">
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.tsx"></script>
  </body>
</html>
```

Figure 6. index.html

The file only contains the doctype declaration, a definition of the characterset and the viewport size being the width of the device it is being viewed on then in the body a script tag pointing to the main.tsx file.

Following into the main.tsx file it is also very simple at setting up the React application. Imported are StrictMode and createRoot from react with StrictMode allowing errors from unsafe methods (GeekForGeeks, 2024) while createRoot is used for React to work inside the browsers DOM. The application is then created in StrictMode loading in from App.tsx while the tailwind styles are imported from being loaded in index.css.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Figure 7. main.tsx

Entering App.tsx everything becomes more complicated as the source of the website is present although not the pages of the website as they are stored as part of the components. The components are imported into the file to be called when appropriate with functions for the co-ordinates and how to load each page as they are not treated as pages in a traditional sense with it all being located on the singular url.

```
function App() {
  const [step, setStep] = useState<'initial' | 'selection' | 'map'>('initial');
  const [selectedLocation, setSelectedLocation] = useState<number | null>(null);
  const [userLocation, setUserLocation] = useState<[number, number] | null>(null);
  const [sortedLocations, setSortedLocations] = useState(locationOptions);
```

Figure 8. Constants definitions

Step is defined as this records the current ‘page’ of the application so it can be loaded correctly while the value of step is set to initial to display the intended landing area. The selectedLoncation constant is created with it’s value always being from the setSelectedLocation function, the default value of this is also set to null as no place has been selected yet. userLocation is then defined in a very similar way from the setUserLocation function with the value being 2 numbers allowing for co-ordinates to be present. The sortedLocations will have all the possible courses present ranked by their co-ordinates proximity to the user’s co-ordinates.

```
useEffect(() => {
  if ('geolocation' in navigator) {
    const watchId = navigator.geolocation.watchPosition((position) => {
      const userCoords: [number, number] = [
        position.coords.latitude,
        position.coords.longitude
      ];
      setUserLocation(userCoords);

      // Sort locations by distance from user
      const locationsWithDistance = locationOptions.map(location => {
        const fullLocation = locations.find(loc => loc.id === location.id);
        if (!fullLocation) return { ...location, distance: Infinity };

        const distance = calculateDistance(
          userCoords[0],
          userCoords[1],
          fullLocation.coordinates[0],
          fullLocation.coordinates[1]
        );

        return {
          ...location,
          distance
        };
      });

        const sorted = locationsWithDistance.sort((a, b) => a.distance - b.distance);
        setSortedLocations(sorted);
      },
      (error) => {
        console.error('Error watching position:', error);
      },
      {
        enableHighAccuracy: true,
        maximumAge: 0,
      }
    );
```

Figure 9. Constant User Positions

The users position is constantly updated utilising the watchPosition function present within Window.navigator object in Web API. If geolocation isn't present within the API the position never tries to be set meaning there isn't an error with could occur. The userCoordinates are set by the function setUserLocation after being cast into userCoords. The watchID is also taken here so it can be checked how old the reading is so it can be updated at a consistent rate with will not be noticeable to the user but this present pulling new data every millisecond causing severe data usage and battery issues for the end user.

The location selection function is then created as this needs to be defined before the run-through of the dummy html to follow so it can be included. The variable id is created as a number with the selected location being the item in

the list and setting the screen to be the final map screen. The constant of selectedLocationData is also created getting the every value associated with the id of the location for it all to be displayed properly.

```
const handleLocationSelect = (id: number) => {
  setSelectedLocation(id);
  setStep('map');
};

const selectedLocationData = locations.find(loc => loc.id === selectedLocation);
```

Figure 10. Location Handling

The overall window is adjusted to be shown as the entire width of the screen vertically and no more with the background being set as the predecided #4B4B4B. All the items to follow will be centred both vertically and horizontally within the divider with 6 pixels padding to the outside of the divider. Another div defines itself to use the full width of the previous divider with items horizontally aligned to take up that full width. The next line checks the value of step and if it is initial a divider with the contents vertically centred is made. Within that a header is created of type h1, the text colour is #D9D9D9 with the text size being 3xl which corresponds to 1.875rem or 30px (tailwindcss.com, n.d.), the text is then made bold as well. The second item is the button which is defined in components, when the button is activated it does change the step to the selection screen as well.

```
return (
  <div className="min-h-screen bg-[#4B4B4B] flex items-center justify-center p-6">
    <div className="w-full flex justify-center">
      {step === 'initial' && (
        <div className="flex flex-col items-center">
          <h1 className="text-[#D9D9D9] text-3xl font-bold mb-8">Course Selection</h1>
          <SelectionButton onClick={() => setStep('selection')} />
        </div>
      )}
```

Figure 11. Initial Screen code

In the course selection third of the application a divider is created with a maximum width of 56rem equivalent of 896px but if the screen is narrower that this it automatically compresses it within the screen due to the responsive design nature of tailwind. Then on the screen there is a Back to start button which sets the step back to initial should the user want to navigate backwards. With a heading telling the user to select a golf course next this text will be semibold to emphasise it to the user. The following divider is to set the bounds of where the courses which can be selected are placed. This has a manually defined maximum height of calc(3.5*theme(height.48) to allow just over 3 full courses to be visible with the rest being scrollable for the user to find one further afield. The Location List has all the locations in in their sorted manner with the user location so it can give the distance to each one. When it is pressed

the location handler is called to pass the correct information on to the map screen.

```
{step === 'selection' && (
  <div className="space-y-4 overflow-hidden max-w-4xl">
    <button
    onClick={() => setStep('initial')}
    className="text-[#D9D9D9] hover:underline flex items-center gap-2"
    >
      ← Back to start
    </button>
    <h2 className="text-[#D9D9D9] text-xl font-semibold mb-4">Select a Golf Course</h2>
    <div className="overflow-auto max-h-[calc(3.5*theme(height.48))] p-8">
      <LocationList
        locations={sortedLocations}
        userLocation={userLocation}
        onSelect={handleLocationSelect}
      />
    </div>
  </div>
)}
```

Figure 12. Selection Screen code

The third and final screen is the map itself and the functionality associated with it. There are responsive design elements incorporated throughout centring the content on the screen. The first major element take the user backwards to the selection screen itself. The next feature lists the name and description of the selected course and sizing of the text itself, the map itself is then displayed with it centring on the courses location but with the users location also marked as well. This allows the distance measuring to occur within the program itself as well as conveying the relevant information. The screens finish with the Next 5 hours of weather information being displayed, this is done by also passing in latitude and longitude measurements with all the data being sourced from Open-Meteo.

```
{step === 'map' && selectedLocationData && (
  <div className="min-h-screen bg-[#4B4B4B] flex flex-col items-center p-6 relative">
    <div className="w-full max-w-4xl space-y-4">
      <button
        onClick={() => setStep('selection')}
        className="text-[#D9D9D9] hover:underline flex items-center gap-2"
      >
        ← Back to selection
      </button>

      <div className="bg-[#D9D9D9] p-4 rounded-lg mb-4">
        <h2 className="text-[#4B4B4B] text-xl font-semibold">{selectedLocationData.name}</h2>
        <p className="text-[#4B4B4B] opacity-75">{selectedLocationData.description}</p>
      </div>
      <LocationMap location={selectedLocationData} userLocation={userLocation} />
    </div>

    <div className="bg-white shadow-lg p-4 rounded-lg w-full max-w-7xl mt-4 flex flex-col xl:justify-center">
      <h3 className="text-lg font-semibold text-gray-800">Next 5 Hours</h3>
      <div className="w-full mb-4 flex flex-col gap-4 xl:gap-x-8 xl:justify-between">
        <WeatherInfo
          latitude={selectedLocationData.coordinates[0]}
          longitude={selectedLocationData.coordinates[1]}
        />
      </div>
      <h4 className="text-lg font-semibold text-gray-800">Data from Open-Meteo</h4>
    </div>
  </div>
)}
```

Figure 13. Map Screen code

There are 2 typescript types which I have manually defined. The first defined type is Location and is the courses location information. It contains an id number, the course name, the coordinates of the course, a description and an image url to be displayed. All of this information is manually inputted and displayed within the selection screen to aid the user's choice of which course they are playing. The second type is LocationOption which are the values which are being carried forward so the coordinates aren't continues through when they are no longer needed.

```
export interface Location {
  id: number;
  name: string;
  coordinates: [number, number];
  description: string;
  image: string;
}

export interface LocationOption {
  id: number;
  name: string;
  description: string;
  image: string;
  distance?: number;
}
```

Figure 14. Manual Data Types

The data which follows these types is present in the data.ts file. This is an organised structured set of data. The id's are sequential with all the details

following. There are 6 courses present on the system and they are Carholme Golf Club, Lincoln Golf Club, Sharpley Golf, Woodhall Spa both the Hotchkin and the Bracken and Market Rasen & District Golf Club. These were chosen as they were potential testing locations for both data collection and functionality testing. All images have been query formatted to a height of 80px and a width of 400px to ensure they fit correctly.

```
export const locations: Location[] = [
  {
    id: 1,
    name: "Carholme Golf Course",
    coordinates: [53.234158, -0.560261],
    description: "A beautiful parkland golf course in Lincoln",
    image: "https://carholmegolfclub.co.uk/wp-content/uploads/2024/12/20240505_153300-scaled.jpg?auto=format&fit=crop&q=80&w=400"
  },
  {
    id: 2,
    name: "Lincoln Golf Club",
    coordinates: [53.303818, -0.737191],
    description: "Historic 18-hole golf course established in 1891",
    image: "https://www.lincolngc.co.uk/wp-content/uploads/2024/11/DJI_0701-1400x994.webp?auto=format&fit=crop&q=80&w=400"
  },
  {
    id: 3,
    name: "Sharpley Golf",
    coordinates: [54.844980, -1.404323],
    description: "Northeast England's Leading Golf Course",
    image: "https://www.sharpleygolf.co.uk/public/uploads/Sharpley.13.09.19.AM.MKIV.11.2.jpg?auto=format&fit=crop&q=80&w=400"
  },
  {
    id: 4,
    name: "Woodhall Spa - Hotchkin",
    coordinates: [53.155399, -0.204674],
    description: "Hotchkin Course - 58th Best Course on the Planet",
    image: "https://cdn.sanity.io/images/arkm1m4f/production/5e294c7692421777ec2a2076103177083cfa9521-1600x1067.jpg?auto=format&fit=crop&q=80&w=400"
  },
  {
    id: 5,
    name: "Woodhall Spa -  Bracken",
    coordinates: [53.156085, -0.206711],
    description: "A contemporary classic and the perfect complement to The Hotchkin Course",
    image: "https://images.contentstack.io/v3/assets/blt99dd26276e65134a/blt626e0b466e380ae2/5bf486e7dafd18fa6a7037bb/hotchkin-9th-green.jpg?auto=format&fit=crop&q=80&w=400"
  },
  {
    id: 6,
    name: "Market Rasen & District Golf Club",
    coordinates: [53.376843, -0.303712],
    description: "One of Lincolnshire's Finest",
    image: "https://www.marketrasengolfclub.co.uk/wp-content/uploads/2024/02/4th-fairway-2300x1300.jpg?auto=format&fit=crop&q=80&w=400"
  }
];
```

Figure 15. data.ts

5.2 Components

There are 4 different components in the program with 1 creating in initial selection button, 1 displaying the courses available for selection and 2 aiding the mapping screen to display all the information they need to with one for the map itself and the other for the weather information.

The initial screen is very simple with it containing a button with a MapPin as decoration on the button. There are also no props needed as the onClick is handled in App.tsx as no further processing is needed other than changing a singular value.

```
interface SelectionButtonProps {
  onClick: () => void;
}

export const SelectionButton: React.FC<SelectionButtonProps> = ({ onClick }) => {
  return (
    <button
      onClick={onClick}
      className="flex items-center gap-2 px-6 py-3 bg-[#D9D9D9] text-[#4B4B4B] rounded-lg hover:bg-opacity-90 transition-all"
    >
      <MapPin className="w-5 h-5" />
      <span className="font-medium">Course Selection</span>
    </button>
  );
};
```

Figure 16. SelectionButton.tsx

For data to be displayed on the interface a Props class has to be defined to hold the data which will be shown on the screen. Within the list of locations this is created as LocationListProps containing locations which in turn contains the entire of the LocationOption class data, with the userLocation being 2 numbers corresponding to the latitude and the longitude.

The first 2 divs define the total area for all the buttons to cover with then all the locations being mapped through the .map attribute. It is all placed within a button with the key being the id of the specific location so each key is unique to the button and on the button being clicked the location id is passed to the function defined in App.tsx which is the handleLocationSelect function. The background of the button is the image provided of the course with, if the alt text being the locations name. It is made so the the text is centred correctly and the image expands within the button when it is being hovered over. There is also a MapPin placed on the button as a purely design element.

The distance to the course from the user is also given on the button. At this point the distance itself has already been calculated and stored in LocationOption so it is in LocationListProps. First we check if it is present so we can actually display the value. We then take this value and round it knowing the actual value may not be a cardinal number. This is followed by telling the user this is in yards. It can lead to amusing situations of a course being 10000 yards away but with calculations being done in yards for the more important distance calculations this is a fair compromise.

```
interface LocationListProps {
  locations: LocationOption[];
  userLocation: [number, number] | null;
  onSelect: (id: number) => void;
}

export const LocationList: React.FC<LocationListProps> = ({ locations, userLocation, onSelect }) => {
  return (
    <div className="w-full max-w-2xl space-y-4">
      <div className="space-y-4">
        {locations.map((location) => (
          <button
            key={location.id}
            onClick={() => onSelect(location.id)}
            className="w-full group overflow-hidden rounded-lg hover:ring-2 hover:ring-[#D9D9D9] transition-all"
          >
            <div className="relative">
              <img
                src={location.image}
                alt={location.name}
                className="w-full h-48 object-cover group-hover:scale-105 transition-transform duration-300"
              />
              <div className="absolute inset-0 bg-black bg-opacity-40 group-hover:bg-opacity-30 transition-all">
                <div className="absolute bottom-0 left-0 right-0 p-4 bg-gradient-to-t from-black to-transparent">
                  <div className="flex items-center justify-between">
                    <div className="flex items-center gap-2">
                      <MapPin className="w-5 h-5 text-[#D9D9D9]" />
                      <h3 className="font-medium text-[#D9D9D9] text-lg">{location.name}</h3>
                    </div>
                    {'distance' in location && userLocation && (
                      <span className="text-[#D9D9D9] text-sm">
                        {Math.round((location as any).distance)} yards away
                      </span>
                    )}
                  </div>
                  <p className="text-sm text-[#D9D9D9] mt-1">{location.description}</p>
                </div>
              </div>
            </div>
          </button>
        ))}
      </div>
    </div>
  );
};
```

Figure 17. LocationList.tsx

The map screen however, has much more complexity due to it's increased interactive functionality. First the style of map pin is standardised as different browsers would have different default pins and the ones used are from cloudflare as png assets. The interface props are then defined with the users location and the selected location needing to be placed onto the screen in this section.

```
delete (L.Icon.Default.prototype as any)._getIconUrl;
L.Icon.Default.mergeOptions({
  iconRetinaUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-icon-2x.png',
  iconUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-icon.png',
  shadowUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-shadow.png',
});

interface LocationMapProps {
  location: Location;
  userLocation: [number, number] | null;
}
```

Figure 18. Pin and MapProps

All the information based on the map and the selected location need to be displayed so to work out the information to be displayed functions have been created using the MapEvents react component with a prop of userLocation which itself is a tuple. Two values are defined with clickedLocation being the value of setClickedLocation and distance being the value of setDistance.

Then useMapEvents is utilised from react leaflet, the click on the map is stored as e, the lat and long are then pulled out of e. If the user has a known location calculateDistance is used with the users lat and long and the clicked lat and long passed in with the result being utilised by setDistance to update the distance state.

This distance value is displayed at the midpoint of the line and as the lat and long of the 2 points are known they can be added together before being divided by 2 to give the midpoints lat and long values with the output being stored in getMidpoint.

```
function MapEvents( {userLocation} : {userLocation: [number, number] | null} ) {
  const [clickedLocation, setClickedLocation] = useState<[number, number] | null>(null);
  const [distance, setDistance] = useState<number | null>(null);

  const map = useMapEvents({
    click(e) {
      const { lat, lng } = e.latlng;
      setClickedLocation([lat, lng]);

      if (userLocation) {
        const dist = calculateDistance(
          userLocation[0],
          userLocation[1],
          lat,
          lng
        );
        setDistance(dist);
      }
    },
  });

  const getMidpoint = (point1: [number, number], point2: [number, number]): [number, number] => {
    return [
      (point1[0] + point2[0]) / 2,
      (point1[1] + point2[1]) / 2
    ];
  };
```

Figure 19. MapEvents

This is followed by a return which is the rendered tsx for mapEvents. If the userLocation has value there is a marker placed at that point in the style defined at the top of the file. A marker is then placed at the clicked location for the same reason in the same style. A line is then drawn between the 2 points to allow the user to see any hazard in the way so they can orientate themselves in real life to the map and direction they which to go. During this section another marker is placed at the calculated midpoint but it's design is text with the

calculated distance between the two points. The label itself is a dark semi-transparent background with the text being white.

```
return (
  <>
    {userLocation && (
      <Marker position={userLocation}>
        <Popup>
          <div className="text-center">
            <p className="font-medium">Your Location</p>
            <p className="text-sm mt-1">
              {userLocation[0].toFixed(6)}, {userLocation[1].toFixed(6)}
            </p>
          </div>
        </Popup>
      </Marker>
    )}
    {clickedLocation && (
      <>
        <Marker position={clickedLocation}>
          <Popup>
            <div className="text-center">
              <p className="font-medium">Selected Point</p>
              <p className="text-sm mt-1">
                {clickedLocation[0].toFixed(6)}, {clickedLocation[1].toFixed(6)}
              </p>
            </div>
          </Popup>
        </Marker>
        {userLocation && distance && (
          <>
            <Polyline
              positions={[userLocation, clickedLocation]}
              pathOptions={{ color: 'white', weight: 2 }}
            />
            <Marker
              position={getMidpoint(userLocation, clickedLocation)}
              icon={L.divIcon({
                className: 'distance-label',
                html: `<div class="bg-black bg-opacity-50 text-white px-2 py-1 rounded text-sm whitespace-nowrap">
                  ${Math.round(distance)} yards
                </div>`,
              })}
            />
          </>
        )}
      </>
    )}
  </>
);
```

Figure 20. Map Render

The final section of LocationMap.tsx is the html itself which is returned with a bar above the map listing what the user can do such as clicking to get the other point, and that their location is show along with all distances being in yards. This is followed by the map itself which centres on the course itself, the satellite map is from arcgis online with data provided by esri who are credited accordingly.

```
export const LocationMap: React.FC<LocationMapProps> = ({ location, userLocation }) => {
  return (
    <div className="space-y-4">
      <div className="flex items-center gap-4 text-[#D9D9D9] text-sm">
        <div className="flex items-center gap-2">
          <MapPin className="w-4 h-4" /> Click to get coordinates
        </div>
        <div className="flex items-center gap-2">
          <Crosshair className="w-4 h-4" /> Your location will be shown
        </div>
        <div className="flex items-center gap-2">
          <Ruler className="w-4 h-4" /> Distance in yards
        </div>
      </div>
      <div className="w-full h-[500px] rounded-lg overflow-hidden">
        <MapContainer
          center={location.coordinates}
          zoom={15}
          style={{ height: '100%', width: '100%' }}
        >
          <TileLayer
            url="https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}"
            attribution='&copy; <a href="https://www.esri.com/">Esri</a>'
          />
          <Marker position={location.coordinates}>
            <Popup>
              <div>
                <h3 className="font-medium">{location.name}</h3>
                <p className="text-sm">{location.description}</p>
              </div>
            </Popup>
          </Marker>
          <MapEvents userLocation={userLocation} />
        </MapContainer>
      </div>
    </div>
  );
};
```

Figure 21. Map HTML

The largest component is WeatherInfo.tsx which makes an api request based off the latitude and longitude of the user, parsing the return and then displaying it. First the props are defined as the latitude and longitude as these need to be passed into the react component WeatherInfo. The states of weatherData, loading and error are defined to be the parsed weather info, if the data is being fetched and if there is any fetch error respectively.

useEffect is utilised to update the weather information, the data is fetched asynchronously as once it is present it doesn't need to be fetched again. The fetch is then tried with the return being stored in response. This then has error checking occurring to ensure the returned information is in a valid format to then be processed. A slice is also taken from the current hour to 6 hours in the future as a round of golf shouldn't take longer than that.

```
interface WeatherInfoProps {
  latitude: number;
  longitude: number;
}

const WeatherInfo: React.FC<WeatherInfoProps> = ({ latitude, longitude }) => {
    const [weatherData, setWeatherData] = useState<{ time: string; temperature: number; windSpeed: number; windDirection: number; precipitationProbability: number;}[]>([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);

    useEffect(() => {
      const fetchWeather = async () => {
        setLoading(true);
        setError(null);

        try {
          const response = await fetch(
            `https://api.open-meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}&hourly=temperature_2m,wind_speed_10m,wind_direction_10m,precipitation_probability`
          );

          if (!response.ok) throw new Error('Failed to fetch weather data');

          const data = await response.json();

          const now = new Date();
          const currentHour = now.getHours(); // Gets current hour (0-23)

          const index = data.hourly.time.findIndex((time: string) => {
            return new Date(time).getHours() === currentHour;
        });
```

Figure 22. Weather Request

This data is then taken and formatted with only the required and allowed information being stored. This information is the time, the temperature, the wind speed and direction and the precipitation probability. Anymore information might run up against what is classified as judgement in golf and thus make it incompatible with scoring play, any less could leave the user with unanswered questions. This data is then set into the weatherData state to be displayed to the user on screen.

This then goes into the return of the html code. There is a check setup to display the state of the request if it is either loading or has an error in case there are issues to prevent it displaying with missing data and showing unknown error messages to the user. The weather data is mapped into the code to give a consistent design for each hour and is placed in a way so they display sequentially left to right on the screen. The wind direction is based on the heading given and utilises a down arrow due to how this heading is returned.

```
      const startIndex = index !== -1 ? index : 0;
      const formattedData = data.hourly.time.slice(startIndex, startIndex + 6).map((time: string, i: number) => ({
        time: new Date(time).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
        temperature: data.hourly.temperature_2m[startIndex + i],
        windSpeed: data.hourly.wind_speed_10m[startIndex + i],
        windDirection: data.hourly.wind_direction_10m[startIndex + i],
        precipitationProbability: data.hourly.precipitation_probability[startIndex + i],
      }));

      setWeatherOata(formattedData);
    } catch (err) {
      setError('Could not load weather data');
    } finally {
      setLoading(false);
    }
  };

  fetchWeather();
}, [latitude, longitude]);

return (
    <div class=="bg-white p-4">
        {loading && <p class=="text-gray-500">Loading...</p>}
        {error && <p class=="text-red-500">{error}</p>}
        {!loading && !error && (
            <div class=="mt-2 space-y-4 lg:space-y-0 lg:flex lg:gap-x-8 lg:justify-between">
                {weatherData.map((hour, index) => (
                    <div key={index} class=="p-2 border-b flex flex-col lg:flex-row lg:items-center md:gap-4">
                        <div class=="flex justify-between lg:flex-col lg:items-start lg:mr-4">
                            <span class=="text-gray-600">{hour.time}</span>
                            <span class=="text-gray-800 font-medium">{hour.temperature}°C</span>
                        </div>
                        <div class=="text-sm text-gray-600 flex justify-between lg:flex-col lg:items-start">
                            <span>Wind Speed {hour.windSpeed} km/h</span>
                            <span>Wind Heading {hour.windDirection}°</span>
                            <span class=="inline-block text-xl" style={{ transform: `rotate(${hour.windDirection}deg)` }}> ↓ </span>
                            <span>☂ {hour.precipitationProbability}%</span>
                        </div>
                    </div>
                ))
            </div>
        )
    </div>
```

Figure 23. Parsing and Return

5.3 Utils

There is one utility function that has been created for the application and that is the Haversine equation which measures the distance between 2 points around a sphere. This has been coded following the formula below. To keep in step with mathematical norms delta, theta and lambda have been used to represent angles, and change between 2 values. All points have first been converted to their radian for from degrees as this is the mathematical standard with the diameter of earth being given in metres as the standard scientific unit before being converted into yards to satisfy the standard unit of measurement for distances in golf.

```
export function calculateDistance(
  lat1: number,
  lon1: number,
  lat2: number,
  lon2: number
): number {
  const R = 6371e3; // Earth's radius in metres
  const ϕ1 = (lat1 * Math.PI) / 180;
  const ϕ2 = (lat2 * Math.PI) / 180;
  const Δϕ = ((lat2 - lat1) * Math.PI) / 180;
  const Δλ = ((lon2 - lon1) * Math.PI) / 180;

  const a =
    Math.sin(Δϕ / 2) * Math.sin(Δϕ / 2) +
    Math.cos(ϕ1) * Math.cos(ϕ2) * Math.sin(Δλ / 2) * Math.sin(Δλ / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));

  const metres = R * c;
  return metres * 1.09361; // Convert to yards
}
```

Figure 24. Haversine Equation code

5.4 Hosted

The resulting web app is present at the url https://bartsimho.github.io/CourseManagement being hosted on Github pages with all the code publicly viewable on Github to allow this to run. The initial landing page looks as follows being very simple and clear about how to move onto the following screen.

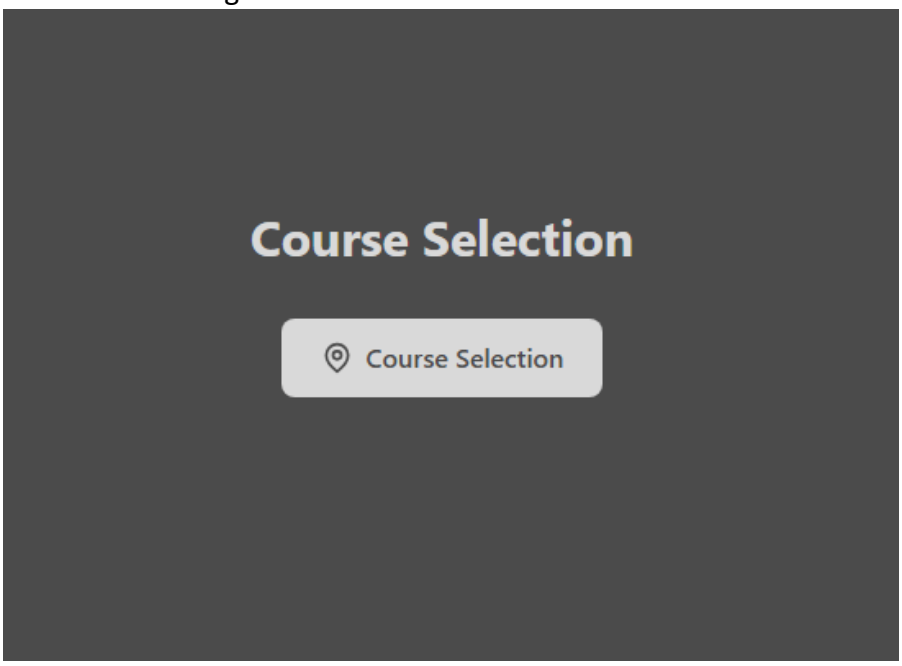

Figure 25. Initial Screen

The following course selection screen is as follows with interactive buttons with images showcasing a shot of each course that can be chosen.

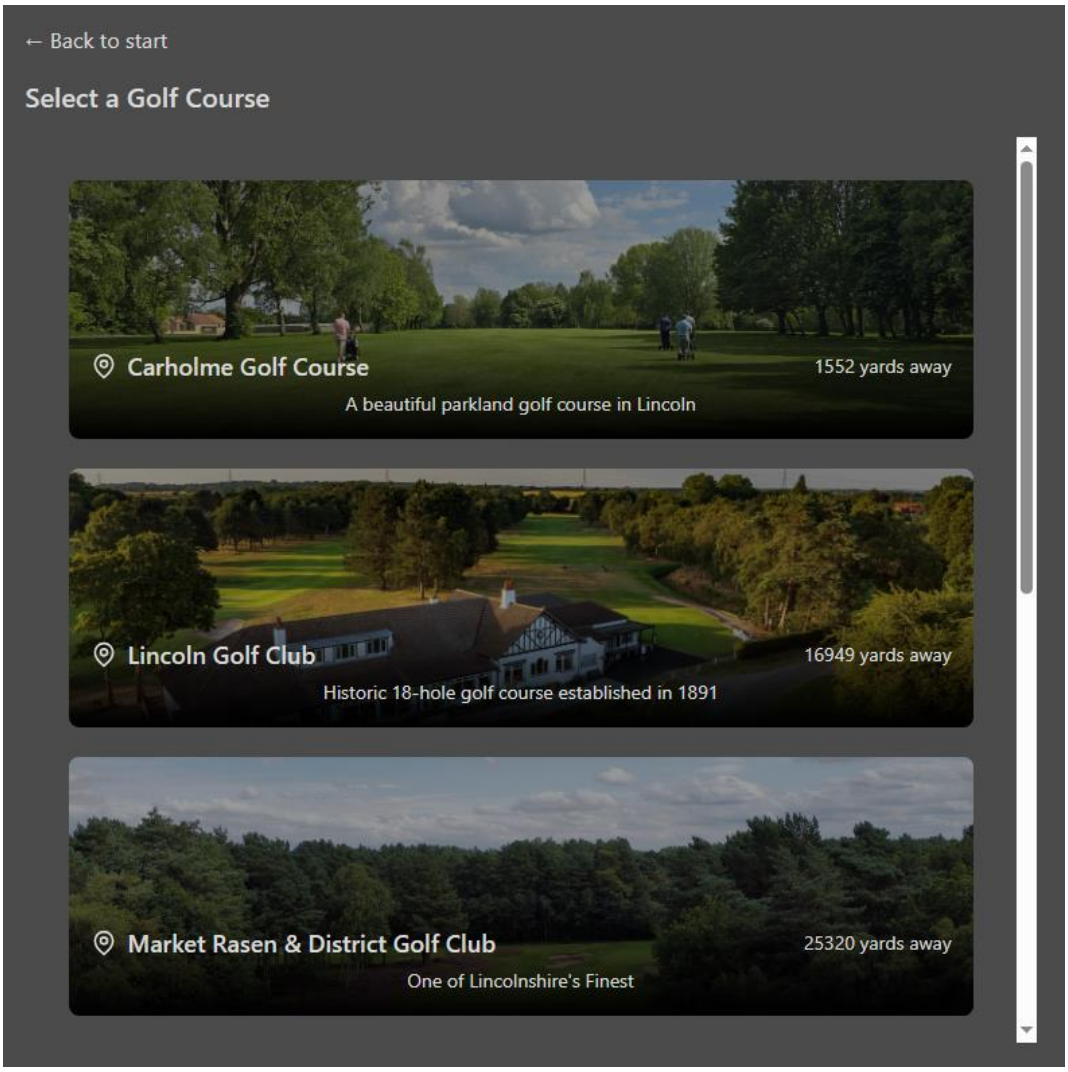

Figure 26. Course Selection Screen

The final screen with the map is more complex with more information present which changes how it is displayed based on how wide the screen it is being viewed is.

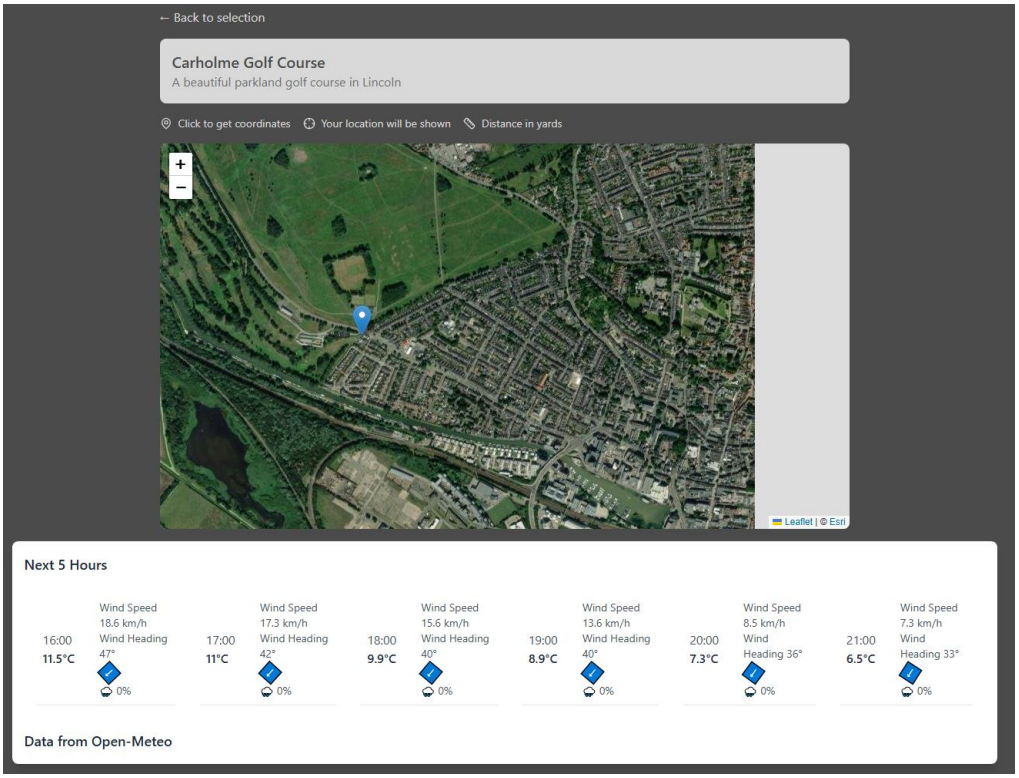

Figure 27. Map Screen

Chapter 6

Results & Discussion

The Testing gave 594 individual pieces of data to be analysed. All the data was structured as per the table below with the Players Handicap, if they used the GPS app or not for the hole, the Par of the hole and their score on the hole.

Handicap	With (1) or Without (0)	Par	Score
37.9	0	3	4
20.0	0	4	5
36.0	0	5	8
15.8	1	3	5
21.0	1	4	6
33.6	1	5	7

Table 2. Sample Data

A mean Handicap was calculated for each type of par and if they used the GPS giving 6 different mean handicaps. These mean handicaps were then placed into the data temporarily as appropriate to allow for a linear regression analysis to take place (Kumari and Yadav, 2018). A linear regression analysis was chosen as it is appropriate for comparing the score on a hole to the par of the hole. With handicaps already adjusted for I could plot all 6 points on a graph to show the difference in scoring throughout these different scenarios tested.

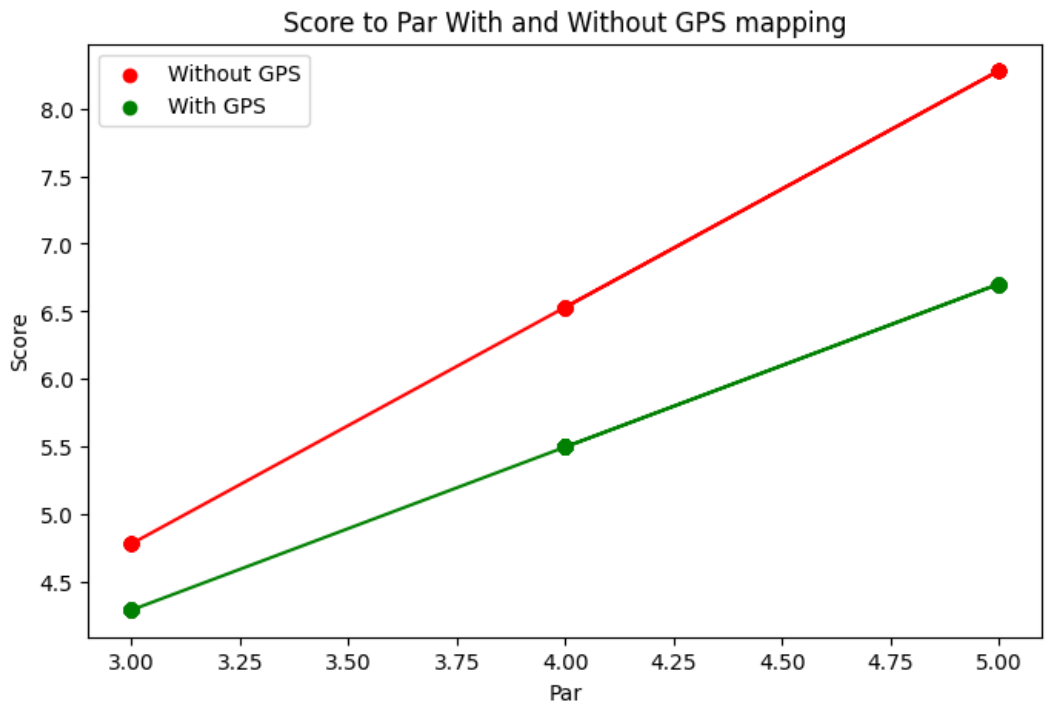

Figure 28. Linear Regression of Scores to Par

As can be seen the scores in all cases where GPS was not used was higher than when GPS was used. The gap between scoring widens as the par of the hole increases potentially suggesting that using GPS becomes more useful the longer the hole is. However this relationship is not fully explored as the distance of a hole was not recorded, this could be a follow-up study which can be conducted.

Although from the visual of the graph is appears that scores improve using GPS data this cannot be statistically confirmed by a look as golf is a sport of high variability in scoring which can be noted by 2 Hole In Ones being achieved throughout the testing. Both of them occurred on holes with a par of 3 however, one was utilising GPS while the other was not. Upon achieving this feat it can be noted that both participants were extremely excited for this feat.

As the actual dataset for each par is uneven on account of utilising myself as a test subject multiple times ,acceptable for this study as the variable being researched in score. There were a total of 43 par 3's played without GPS, 97 par 3's with GPS, 84 par 4's without GPS, 281 par 4's with GPS, 17 par 5's without GPS and 72 par 5's with GPS. All of these data points have been mapped onto Histograms in the figures below.

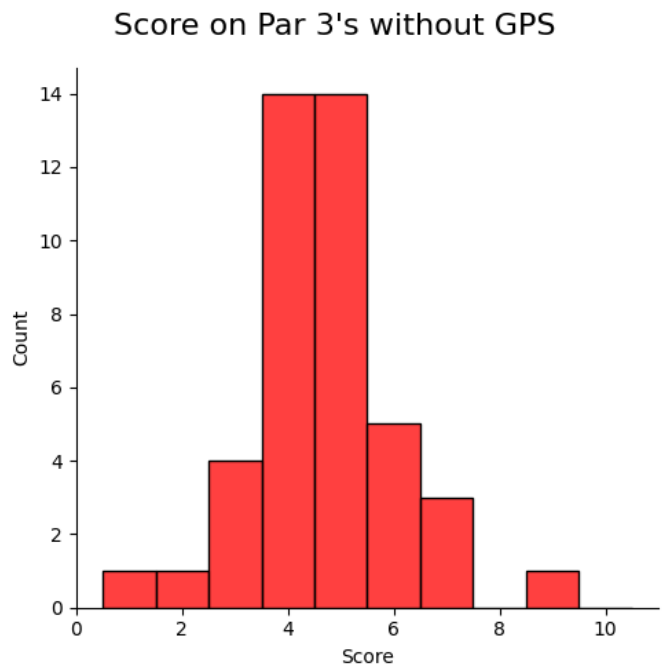

Figure 29. Par 3 Without GPS Histogram

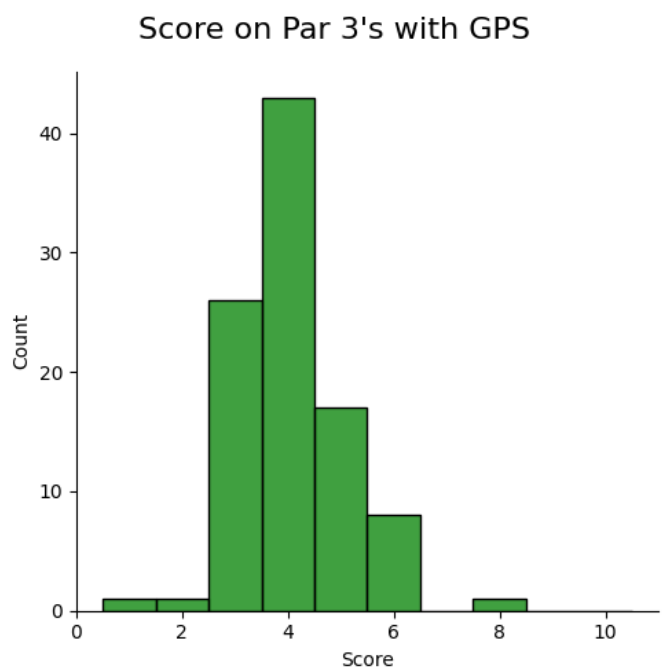

Figure 30. Par 3 With GPS Histogram

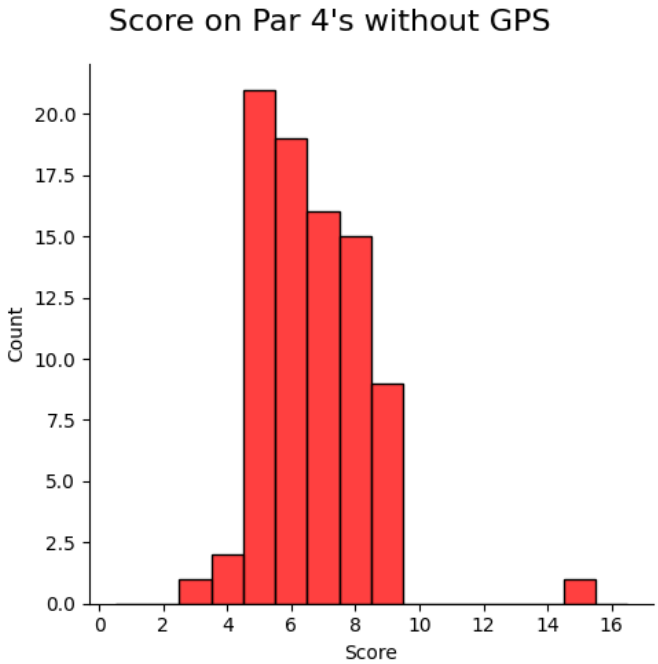

Figure 31. Par 4 Without GPS Histogram

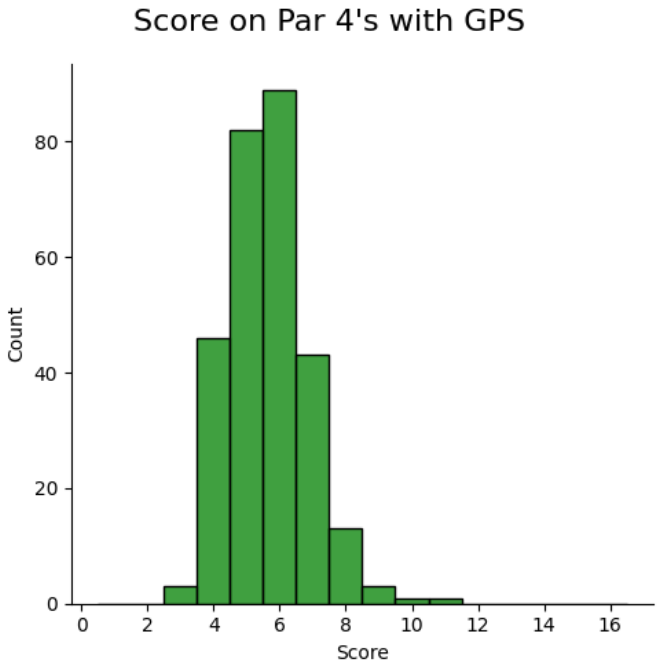

Figure 32. Par 4 With GPS Histogram

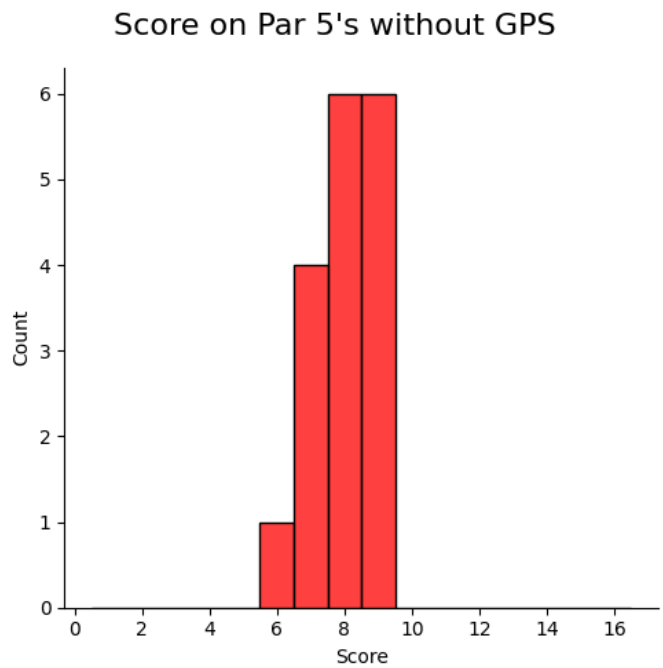

Figure 33. Par 5 Without GPS Histogram

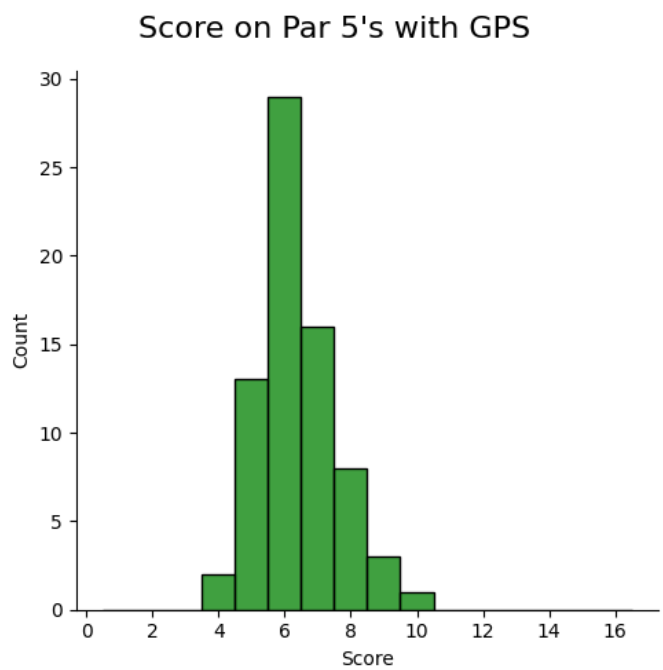

Figure 34. Par 5 With GPS Histogram

These data points do not follow a normal distribution so they do not meet the requirements to perform a T-test on. However, a Mann-Whitney U Test can be performed as it is designed for where the assumption of a normal distribution fails. It is also beneficial as it works well with small sample sizes which I would deem each of the 6 different sections of the data as well as comparing 2 independent groups which in this case are the scores (Sundjaja, Shrestha and Krishan, 2021).

The results of the tests are detailed in the table below but for Par 3's there is a U Value of 2713 and a p-value of 0.00300443, for Par 4's a U Value of 15963 and a p-value of $4.53312259 \times 10^{-7}$, and for Par 5's a U Value of 1038.5 and a p-value of $4.64940359 \times 10^{-6}$. These values suggest a confidence of 99.7%, 99.99995% and 99.99954% respectively.

Par	**U Value**	**P Value**	**Confidence**
3	2713	0.00300443	99.7%
4	15963	$4.53312259 \times 10^{-7}$	99.9999995%
5	1038.5	$4.64940359 \times 10^{-6}$	99.999995%

Table 3. Results from Mann-Whitney U Test

As we might not be able to assume the variance of the distributions Welch's T-Test has also been performed to confirm the confidence margins generated by the Mann-Whitney U Test. This test gives the results of T-statistic of 2.8717 and P-value of 0.0047 for Par 3's, a T-statistic of 5.8515 and P-value of 1.0889×10^{-8} for Par 4's and a T-statistic of 5.19424 and P-value of 1.3375×10^{-6} for Par 5's.

Par	**T Statistic**	**P Value**	**Confidence**
3	2.8717	0.0047	99.53%
4	5.8515	1.0889×10^{-8}	99.9999989%
5	5.19424	1.3375×10^{-6}	99.99987%

Table 4. Results from Welch's T Test

These statistics suggest that the data sets are not that different in their variance but are statistically different with very high confidence margins.

Chapter 7

Conclusion

As a whole the project has ran smoothly with very few instances of hardship when creating the web application after defining the initial question to be answered. However, there were hardships with initial ideas being scrapped for a more fleshed out web application due to hosting constraints as well as deeming them secondary to the research question, while feature may be present in an application for release they do not aid the fundamental premise.

The main success of the project is the confidence margin of the findings with all being at least 99%. This indicates that yes GPS distance applications do aid golfers scoring on a hole, although further research could be done to verify the findings as well with a wider range of test data over a longer period of time.

The main difficulty of the project was completing the testing as it required playing golf and courses were not always the most accommodating. As such there is a thank you to members of courses who could book on and test with me to be able to complete the project.

References

R&A (2022). Official Guide to the Rules of Golf. Cassell.

Golf's Grand Design. (2012). [DVD] United States: PBS.

GPS.gov (2019). GPS.gov: GPS Accuracy. Gps.gov. Available at: https://www.gps.gov/systems/gps/performance/accuracy/.

Sachan, A. (2024). How Mobile Apps Connect to Servers: A Comprehensive Guide. Medium. Available at: https://medium.com/@ankit.sachan/how-mobile-apps-connect-to-servers-a-comprehensive-guide-4af081c12813.

Heap. (n.d.). What is a Tech Stack? Choosing What Goes In Yours. Available at: https://www.heap.io/topics/what-is-a-tech-stack.

Raroque, C. (n.d.). How to Build Your Product's Tech Stack: A+ Guide for 2024. Aloa. Available at: https://aloa.co/blog/tech-stack

Jowett, S. and Zhong, X. (2016). Promoting Performance and Satisfaction Through Quality Golfer-Caddie Relationships. International Journal of Golf Science, 5(2), pp.98–115. doi:https://doi.org/10.1123/ijgs.2015-0016.

Carey, L.M., Stone, J.M. and Lavallee, D. (2021). 'Teamwork Done to a Tee': A Golf Caddie's Perspective on Their Perceived Role and Associated Skills. International Journal of Golf Science, 9(1). Available at: https://www.golfsciencejournal.org/article/28084.

Shaw, R. (2024a) Project Proposal. University of Lincoln

Dithurbide, L. and Neyedli, H. (2019). Trust in Distance Measuring Devices (DMDs) Automation in Golf. International Journal of Golf Science, 7(1). Available at: https://www.golfsciencejournal.org/article/7408

Haskett, W. (2022). The Science of Golf. Simon and Schuster.

Tóth, Á. and Lógó, E. (2018). The Effect of Gamification in Sport Applications. IEEE Xplore. doi:https://doi.org/10.1109/CogInfoCom.2018.8639934.

Shotscope.com. (2023). Free Golf GPS Apps | Best Golf App for Android & iOS. Available at: https://shotscope.com/uk/dashboard-app/golf-gps-app/

ForesightEurope (2024). FSX Play | Foresight Sports Europe. Available at: https://foresightsports.eu/software/play/

Motocaddy. (2024). Motocaddy. Available at: https://www.motocaddy.com/uk/

Mack, A.B., (2015). A Model of Score Minimization and Rational Strategic Behavior in Golf (No. 284466).

Broadie, M., (2011). Putts Gained - Measuring Putting on the PGA Tour

Broadie, M. (2014). Every shot counts : using the revolutionary strokes gained approach to improve your golf performance and strategy. New York Gotham Books.

Shapcott, S. (2021). Golfers' Decision Making on the Golf Course. USGA.

Barzeski, E. and Wedzik, D. (2014). Lowest Score Wins. Analyzr Golf L.L.C.

Broadie, M. (2008). Assessing golfer performance using golfmetrics. Science and

Golf V: Proceedings of the World Scientific Congress of Golf, (34), 253–262.

Elmore, R. and Urbaczewski, A. (2019). Loss Aversion in Professional Golf. SSRN Electronic Journal. doi:https://doi.org/10.2139/ssrn.3311649.

Gnagy, E., Dixon, M., Clingerman, E. and Bartholomew, J. (2015). An Exploration of Strategic Decision Making in Golf: Take a Chance, It's Worth the Risk. International Journal of Golf Science, 4(2), pp.89–109. doi:https://doi.org/10.1123/ijgs.2014-0016.

Sachau, D., Simmering, L. and Adler, M. (2012). A Birdie in the Hand: Asymmetry in Golf Risk Preferences. International Journal of Golf Science, 1(2), pp.81–89. doi:https://doi.org/10.1123/ijgs.1.2.81.

Kirschenbaum, D.S., O'Connor, E.A. and Owens, D. (1999). Positive illusions in golf: Empirical and conceptual analyses. Journal of Applied Sport Psychology, 11(1), pp.1–27. doi:https://doi.org/10.1080/10413209908402948.

Clark, E.C., Burnett, T., Blair, R., Traynor, R.L., Hagerman, L. and Dobbins, M. (2024). Strategies to implement evidence-informed decision making at the organizational level: a rap-id systematic review. BMC health services research (Online), 24(1). doi:https://doi.org/10.1186/s12913-024-10841-3.

W3.org. (2024). Web Content Accessibility Guidelines (WCAG) 2.1.

NVISIA (2020). Agile Methodology 101. Available at: https://www.nvisia.com/insights/agile-methodology.

GeeksforGeeks (2022). What are the advantages of React.js ? GeeksforGeeks. Available at: https://www.geeksforgeeks.org/what-are-the-advantages-of-react-js/.

Caio (2024). React + Vite: why use? DEV Community. Available at: https://dev.to/doccaio/react-vite-why-use-cg2.

Staff, A. (2012). Microsoft TypeScript: the JavaScript we need, or a solution looking for a problem? Ars Technica. Available at: https://arstechnica.com/information-technology/2012/10/microsoft-typescript-the-javascript-we-need-or-a-solution-looking-for-a-problem/.

GitHub (n.d.). About GitHub Pages. GitHub Docs. Available at: https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages.

Shaw, R. (2024b) User Experience Design Assessment Item 1. University of Lincoln

bolt.new. (2024). bolt.new.

Robusto, C.C. (1957). The Cosine-Haversine Formula. The American Mathematical Month-ly, 64(1), p.38. doi:https://doi.org/10.2307/2309088.

MDN Web Docs. (n.d.a). Geolocation.getCurrentPosition(). Available at: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation/getCurrentPosition.

MDN Web Docs. (n.d.b). Geolocation.watchPosition(). Available at: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation/watchPosition.

Google (2019). Measure distances and areas in Google Earth - Computer - Google Earth Help. Available at: https://support.google.com/earth/answer/9010337.

GeekForGeeks (2024). What is StrictMode in React ? GeekForGeeks.

tailwindcss.com. (n.d.). Font Size - Tailwind CSS.

Kumari, K. and Yadav, S. (2018). Linear Regression Analysis Study. Journal of the Practice of Cardiovascular Sciences, 4(1), pp.33–36. doi:https://doi.org/10.4103/jpcs.jpcs_8_18.

Sundjaja, J.H., Shrestha, R. and Krishan, K. (2021). McNemar And Mann-Whitney U Tests.

Shaw, R. (2025) Literature Review. University of Lincoln

Appendix A

UNIVERSITY OF LINCOLN

Ethics reference: UoL2024_10196

CONSENT TO PARTICIPATE IN RESEARCH

Title of Project: An Investigation and Technical Demonstration of a Course Management System for Golfers

Name of Researcher: Ryan Shaw

Please tick/check box

1. I confirm that I have read the information sheet dated 28/10/24 (version 1.0) for the above study. I have had the opportunity to consider the information, ask questions and have
   had these answered satisfactorily. ☐
2. I understand that as I have completed the study anonymously it will not be possible to remove any information I have provided, as you will not be able to identify me in any way. ☐
3. I understand that individuals from the University of Lincoln may look at research data collected during the study, to ensure that the study is conducted appropriately. ☐
4. I understand that the information collected about me will be used to support other research in the future, and may be shared anonymously with other researchers. ☐
5. I agree to take part in the above study. ☐

Figure 35. Consent Form

Participant Information Sheet/Information about the research

(Draft Version 1.0 / Final version 1.0: 28/10/24)

Title of Study: An Investigation and Technical Demonstration of a digital course management system for golfers

We are inviting you to take part in a research study. Before you decide, it is important that you know why we are doing the study and what is involved. Please read the following information carefully.

What is the purpose of the study?

The purpose of this study is to identify if using digital course management software helps golfers improve their score over a hole.

This is relevant as it helps identify if course management aids golfers and if real time distance information helps golfers.

This study will be conducted at various Golf Courses such as Carholme Golf Club and Lincoln Golf Club

Am I eligible to take part?

You are being invited to take part because you are over 18 and have a golf handicap index of under 54.0

Do I have to take part?

Participation is completely voluntary. You should only take part if you want to and choosing not to take part will not disadvantage you in anyway.

What will I be asked to do?

You will be asked to play 8 holes twice and in a random order. The use of the application will alternate with the first hole you shall use the application second while on the second hole you will use the application first. You will be asked to play by the official Rule of Golf as dictated by the R&A as well as any local rules in place at the golf course being used. You will need to record your score on the hole yourself as well as the researcher recording your score as well for you.

Will I be paid expenses for taking part?

You will not be paid to participate in the study and no expenses will be paid either

What are the possible benefits / risks of taking part?

The possible benefits of taking part in this study is access to the application after the study has been completed.

The risks are injury from the physical exertion of playing golf and broken clubs should one of them fail during the study taking place

Will anyone know I have taken part?

The University of Lincoln (UoL) is the sponsor for this study based in the United Kingdom. We will be using information from you in order to undertake this study and will act as the data controller for this study. This means that we are responsible for looking after your information and using it properly.

The study will be confidential unless you reveal that you or someone you know is at serious risk of harm. We will talk about it with you so that we can work out together how best to handle this. You will not be identified in the final report.

Where will my data be stored?

The data obtained from the study will be stored securely on the university OneDrive in password protected files. Only the researcher/researchers will have access to it. Paper copies will be stored in a

Figure 36. Participant Information Sheet (1/2)

secure cabinet/office at the University. The data from this study *may* be put in an Open Access repository for other researchers to use in future research. If so, responses will be anonymised and any personal data (e.g. contact details) will be removed.

What will happen if I don't want to carry on with the study?

You are free to withdraw at any point from this study, without having to give a reason, by contacting the email address you were originally approached on.

If you choose to withdraw from the study, the information you have given us up to this point will be deleted/destroyed. However, once the anonymised data set has been created it will not be possible to remove your anonymised data from the analysis.

What will happen to the results of the research study?

The results of the study will be published in a Dissertation and should you so wish at the time you will be contacted with a copy of the dissertation as well as a summary of the results

Who is organising and funding the research?

This research is being organised by Ryan Shaw at the University of Lincoln

Who has reviewed the study?

All research conducted by the University of Lincoln is looked at by an independent group of people, called a Research Ethics Committee, to protect your rights, dignity and wellbeing. This study has been reviewed and given favourable opinion by a University of Lincoln Research Ethics Committee UoL2024_10196

What if there is a problem?

It is very unlikely that this study would cause you any harm. If you have a concern or a complaint about any aspect of this study, you should ask to speak to the researchers who will do their best to answer your questions. The researchers contact details are given at the end of this information sheet.

If you remain unhappy and wish to complain formally, you can make a formal complaint through the University complaints procedure or by contacting ethics@lincoln.ac.uk

Further information and contact details

Figure 37. Participant Information Sheet (2/2)